Threat Model

Part 1. Stakeholder Analysis

The Secure Thermostat System includes several stakeholders, such as owners, guests, technicians, and potential attackers. Each stakeholder has different goals, capabilities, and access permissions. Identifying these stakeholders helps us understand possible security threats and determine how the system should protect its resources.

2.2.1. Legitimate stakeholders.

1. Homeowner

Role: The Homeowner is the primary administrator and the user with the highest level of application access. The Homeowner is also responsible for the initial security configuration of the system. During the first login, the application requires the default administrator credentials to be replaced with a new username and a strong password. This reduces the risk of unauthorized access through publicly known default credentials.

Goals: The Homeowner wants to maintain a comfortable indoor environment, manage HVAC operation, monitor energy consumption, and control who can access the thermostat.

Capabilities: The Homeowner can manually change the HVAC mode between Off, Heat, Cool, Auto, and Fan and set a target temperature. These operations are implemented through *set_mode()* and *set_target()* in thermostat.rs.

The Homeowner can also create, edit, delete, and activate HVAC profiles. The application includes built-in Home, Away, Sleep, and Vacation profiles. Custom profiles can be deleted, while built-in profiles are protected from deletion.

In addition, the Homeowner can view energy-usage reports showing heating, cooling, and fan runtime. The *energy_report()* function retrieves usage information from the SQLite database.

The Homeowner is responsible for user administration. Through the CLI, they can create Guest accounts with six-digit PINs and Technician accounts with complex passwords and limited access periods. They can also unlock Guest accounts, reset Guest PINs, revoke Guest access, and renew or revoke Technician access.

The *homeowner_menu()* function in cli.rs provides access to these administrative operations.

The Homeowner interacts with the application through an authenticated command-line interface and can monitor the live dashboard, which displays the active HVAC profile, equipment status, indoor and outdoor temperatures, and environmental conditions. Manual temperature adjustments are restricted to the range of 30°F to 100°F. The Homeowner also determines how many hours a Technician account remains authorized to access the system.

2. Technician

Role: The Technician is an authorized user responsible for troubleshooting and maintaining thermostat operation.

Goals: The Technician aims to identify operational problems, examine environmental conditions, adjust HVAC settings when necessary, and verify that the system is functioning correctly.

Capabilities: The Technician can manually change HVAC modes and target temperatures using the same thermostat control functions available to the Homeowner. They can also activate existing HVAC profiles and view energy-usage reports.

The *technician_menu()* function in *cli.rs* additionally provides access to system diagnostics. These diagnostics display the current HVAC equipment state, sensor health, and warnings about elevated carbon monoxide readings.

The diagnostic functionality allows the Technician to examine sensor health, HVAC equipment status, and system safety warnings. For example, the system can identify invalid sensor readings and indicate when heating and cooling have been disabled to prevent unsafe operation. The Technician can also review daily heating, cooling, and fan runtime, as well as cumulative energy-usage totals.

Unlike the Homeowner, the Technician cannot create or manage user accounts or modify saved HVAC profiles through the Technician menu.

Technician access is temporary. When the Homeowner creates a Technician account, the system records an expiration time in the *valid_until* database field. The *require_active_account()* function in *thermostat.rs* checks this expiration before protected thermostat operations. If the authorization period has ended, the function returns ThermostatError::AccessExpired.

Security relevance: A Technician has sufficient permissions to affect heating and cooling behavior. A malicious or compromised Technician could misuse these capabilities during an authorized access period. The expiration mechanism reduces the duration of authorized access, although it does not prevent misuse while the account remains valid.

3. Guest

Role: The Guest is a limited-access user who can interact with the thermostat without administrative privileges.

Goals: The Guest wants to select an appropriate temperature configuration without managing the system or changing its saved settings.

Capabilities: The Guest can view the thermostat dashboard and activate existing HVAC profiles, including built-in and custom profiles. The *guest_menu()* function in cli.rs provides only profile activation and logout options.

When a Guest selects a profile, the *apply_profile()* function in thermostat.rs applies the profile's stored HVAC mode and target temperature to the live thermostat state.

However, the Guest cannot manually change the target temperature or HVAC mode. The *require_homeowner_or_technician()* function explicitly rejects Guest users attempting these operations by returning ThermostatError::NotAllowed.

Guests also cannot access energy-usage reports, manage accounts, or create, edit, or delete HVAC profiles.

Guest authentication uses a six-digit PIN. The *pin_check()* function in auth.rs rejects PINs containing nonnumeric characters, six identical digits, or strictly sequential ascending or descending digits.

Security relevance: Although Guests have limited permissions, they can still affect HVAC operation by activating profiles. A malicious Guest could repeatedly switch between existing profiles, potentially disrupting the desired temperature configuration. Guest permissions therefore represent limited control rather than read-only access.

2.2.2 Adversarial Stakeholders

1. External attacker

Role: An unauthorized individual attempting to access the thermostat without a legitimate account.

Goals: The attacker may seek to obtain unauthorized control of the thermostat, change temperature settings, access stored information, or disrupt system operation.

Potential capabilities:

An external attacker with access to the application's login interface could attempt to guess usernames, passwords, or Guest PINs.

The auth_user() function in database.rs verifies supplied credentials and records failed authentication attempts. After five unsuccessful attempts against an existing account, the application sets a ten-minute lockout period using the locked_until field.

The attacker may also attempt to obtain valid credentials through means outside the application, such as credential theft. However, the supplied code does not establish that the application provides a network-facing interface or remote authentication service.

Security relevance: Unauthorized access could allow the attacker to impersonate a legitimate stakeholder. The consequences would depend on the privileges associated with the compromised account.

2. Malicious Guest or Technician

Role: A legitimate user who intentionally attempts to perform actions beyond their assigned permissions or misuses permitted functions.

Goals: A malicious Guest may attempt to change restricted thermostat settings, while a malicious Technician may attempt to interfere with HVAC operation or continue using the system after authorization expires.

Potential capabilities:

A malicious Guest already has valid credentials and can activate existing profiles. They may attempt to bypass restrictions on manual HVAC control or profile modification.

A malicious Technician can legitimately modify HVAC modes and target temperatures while their account is active. They may misuse this access by selecting inappropriate settings or repeatedly changing configurations.

The application restricts these actions through role checks and account-validity checks. In particular, set_mode(), set_target(), and energy_report() verify both role permissions and account status.

Security relevance: This threat illustrates the difference between unauthorized actions and abuse of legitimate privileges. Role-based access control can prevent a Guest from performing Technician operations, but it cannot by itself determine whether an authorized Technician's temperature adjustment is malicious.

3. Attacker with Compromised Credentials

Role: An individual who obtains a legitimate user's username and password or PIN.

Goals: The attacker aims to impersonate an authorized user and perform operations available to that account.

Potential capabilities:

If the attacker obtains Homeowner credentials, they may gain access to administrative operations, including account creation, account revocation, and thermostat configuration.

Compromised Technician credentials may allow HVAC adjustments, energy-report access, and diagnostics during the Technician's valid access period.

Compromised Guest credentials would provide more limited access, primarily allowing activation of existing profiles.

The application uses Argon2 password hashing through password_hasher() and verifies passwords through password_hash_verify() in auth.rs. This protects stored passwords against direct plaintext disclosure but does not prevent someone from using a password they have already stolen.

Security relevance: The severity of credential compromise depends on the user's role. The Homeowner account is particularly sensitive because it controls both thermostat configuration and user administration.

4. Attacker with Local File or Database Access

Role: An individual who has obtained unauthorized access to files used by the application.

Goals: The attacker may attempt to modify thermostat settings, manipulate user-account records, falsify environmental inputs, or disrupt the simulation.

Potential capabilities:

The system stores account information in SQLite, including usernames, password hashes, roles, failed login attempts, lockout times, and Technician expiration times.

The thermostat_state table stores the current HVAC mode, target temperature, indoor temperature, active profile, and equipment output. The energy_usage table stores accumulated heating, cooling, and fan runtime.

The application also reads simulated environmental inputs from a file, whose path can be configured using the SIM_ENV_FILE environment variable.

An attacker with sufficient filesystem permissions could attempt to modify these resources directly, bypassing the normal CLI workflow.

Security relevance: Application-level role checks do not necessarily protect against direct modification of database or configuration files. This makes local file permissions and data integrity important security considerations.

Part 2. Asset enumeration

Description

The Smart Thermostat CLI contains several assets that require protection to maintain secure authentication, enforce user permissions, preserve accurate thermostat operation, and ensure system availability. The following assets were identified based on the Rust implementation, SQLite database structure, and stakeholder analysis.

Asset & Description	**Security Properties (CIA)**	**Threat Exposure**	**Code reference**
1. **User credentials and Authentification records** - Usernames, Argon2 passwor/ PIN hashes, failed login attempts, lockout timestamps stored in SQLite.	C: Protect password hashes from disclosure and offline cracking attempts. I: Prevent unauthorized changes to credentials and lockout records A: Keep authentication data accessible for legitimate logins	Adversaries may guess PIN or passwords. Attackers with stolen credentials may impersonate users. Adversaries with database access may extract hashes or alter authentication records.	*auth.rs: password_hasher(), password_hash_verify(); database.rs: auth_user(), users table.*
2. **User Roles and Access Permissions -** Homeowner, Technician, and Guest Role assignments, including Technician access-expiration timestamps.	C: Limit unnecessary disclosure of account info. I: Prevent unauthorized privilege changes or extensions of access. A: Ensure that authorization data remains available for permission checks.	Malicious Guests or Technicians may attempt privilege escalation. Attackers with database access may change roles or expiration timestamps. Compromised Homeowner credentials may permit unauthorized account management.	*auth.rs: Role; thermostat.rs: require_active_account(), require_homeowner_or_technician(); database.rs: valid_until.*
3. Thermostat Configuration and State - Current HVAC mode,	C: Protect information about indoor conditions. I: Prevent unauthorized	Compromised Homeowner or Technician accounts	*thermostat.rs: set_mode(), set_target(),*

target temperature, indoor temperature, active profile, and equipment status stored in SQLite	changes to temperature settings and operating modes. A: Maintain access to current settings for HVAC operation.	may modify settings. Guests can change active settings by selecting profiles. Attackers with database access may directly manipulate stored values.	*apply_profile(), thermostat_state table.*
4. HVAC profiles - Built-in Home, Away, Sleep, and Vacation profiles, custom profiles	C: Protect potentially sensitive household preferences. I: Prevent unauthorized profile modification or deletion. A: Ensure profiles remain available for activation.	A compromised Homeowner may modify or delete custom profiles. Malicious Guests may repeatedly activate different profiles. Attackers with database access may alter profile records.	*profiles.rs: profile creation, modification, deletion, activation; SQLite profiles table.*
5. Environmental and sensor data - outdoor/ indoor temperatures, etc.	C: Protect information about indoor environmental conditions. I: Maintain accurate readings for safe control decisions. A: Ensure valid readings are available for thermostat operation.	Attackers with local file access may manipulate simulated environmental inputs. Incorrect readings may trigger a sensor fault, disable heating and cooling, or affect control decisions.	*sensors.rs, sim.rs, thermostat.rs; SIM_ENV_FILE.*
6. HVAC Control Logic	C: Low confidentiality requirement for the algorithm itself. I: Preserve correct control decisions and sensor-fault handling. A: Maintain reliable HVAC control functionality.	Attackers with access to the application executable or source code may attempt to alter control behavior. Manipulated sensor readings may affect decisions.	*hvac.rs: decide(), DEADBAND_F, Output.*

7. Energy-Usage Records - Daily and cumulative heating, cooling, and fan runtime stored in SQLite.	C: Restrict access to authorized users because usage patterns may reveal household activity. I: Prevent falsification of runtime records. A: Keep historical records accessible for reporting.	Compromised Homeowner or Technician accounts may access reports. Attackers with database access may alter or delete usage records.	*thermostat.rs: energy_report(), energy_usage table.*
8. Application Configuration - Database path, initial administrator password, simulation speed, and environmental input-file path.	C: Protect sensitive values, particularly the initial administrator password. I: Prevent unauthorized configuration changes. A: Ensure valid settings are available during startup and operation.	Attackers with local configuration access may obtain default credentials, redirect database access, or modify simulation parameters.	*.env, database.rs: DB_NAME, DEFAULT_HOMEOWNER_PASSWORD ; thermostat.rs: SIM_SPEED, SIM_ENV_FILE.*
9. System Availability- The application's ability to authenticate users, maintain thermostat state, process commands, and run the HVAC simulation.	C: Not a primary requirement for availability itself. I: Preserve consistent system state. A: Ensure continuous access to essential thermostat functions.	External attackers may deliberately trigger account lockouts. Attackers with local file access may corrupt the database or configuration, disrupting normal operation.	*main.rs, cli.rs, database.rs, thermostat.rs: tick_now().*

Part 3. STRIDE Threat Analysis

This table identifies the system's main assets, their security requirements, potential STRIDE threats, and the security mechanisms implemented or recommended to protect them.

Asset	**Security Needs**	**Main Threats**	**Existing Defenses / Limitations**
User Credentials (Passwords and PINs)	Confidentiality, Integrity	S: Attackers may impersonate legitimate users by guessing or stealing passwords and PINs. I: Unauthorized access to stored password hashes may allow offline password attacks.	Passwords are hashed using Argon2. Passwords must meet complexity requirements, while six-digit PINs cannot contain identical or fully sequential digits. Accounts are temporarily locked after five failed login attempts. Database file protection needs verification.
Default Homeowner account	Confidentiality, Integrity	S: Someone uses the publicly documented default username and password to gain Homeowner access. E: An attacker obtains full administrative privileges before the legitimate Homeowner completes initial setup.	The system creates a default admin account during initialization. On first login, the Homeowner must replace the default username and password before accessing the Homeowner menu.
SQLite Database	Confidentiality, Integrity, Availability	T: An attacker with filesystem access may modify user roles, credentials, or account expiration dates. I: Unauthorized database access may expose sensitive account information. D: Database corruption or deletion may prevent the application from functioning.	Parameterized SQL queries reduce SQL injection risks. Database initialization creates the required tables. Additional filesystem access restrictions and backup mechanisms should be considered.
Authentication and account lockout	Integrity, Availability	S: An adversary repeatedly attempts to authenticate using incorrect or stolen credentials D: An attacker intentionally triggers account lockout to prevent legitimate users from logging in.	auth_user() verifies credentials and tracks failed attempts. After five unsuccessful attempts, the account is locked for ten minutes. Lockout abuse remains a potential limitation.
User roles (Homeowner,	Integrity	E: A Guest attempts to execute Homeowner-only commands,	Role defines three permitted roles.

Technician, Guest)		such as creating accounts or resetting credentials. A Technician attempts to access functions outside their permissions.	str_to_emum() converts database role strings into valid role values. cli.rs provides separate menus according to the authenticated user's role. Authorization within individual operations must also be verified.
Technician accounts and expiration	Integrity	E: A Technician attempts to continue accessing the system after their authorized period ends or their account is revoked.	User includes valid_until. auth_user() checks account expiration against the current UTC time. revoke_technician() sets valid_until = NULL, which may bypass the expiration check and requires testing.
Account management (creation, reset, revocation)	Integrity, Confidentiality	T: Unauthorized changes to passwords, PINs, or account information. E: A lower-privileged user attempts to create or modify accounts. R: Users deny changing credentials or revoking access.	cli.rs places account management in the Homeowner menu. Database functions such as update_cred(), reset_guest_pin(), and revoke_guest() perform the corresponding changes. Persistent audit logging has not been verified.
Temperature profiles	Integrity, Availability	T: Users modify or activate profiles without permission, changing the intended temperature settings. E: Guests attempt to access profile-management functions reserved for other roles.	cli.rs provides role-specific access to profile operations. Profile creation, selection, and validation are handled through the profiles and thermostat modules, which require further review.
Sensor data and environmental input (environment.txt)	Integrity, Availability	T: Someone modifies environmental input values to produce inaccurate temperature or weather readings. S: Manipulated values appear to originate from legitimate sensor input. D: Missing or malformed input disrupts sensor processing.	main.rs calls sensors::create_template_if_missing() to create an optional environmental override file. If template creation fails, the program reports the error and uses generated weather. Sensor value validation requires verification.

Carbon monoxide safety system	Integrity, Availability	T: Manipulated CO readings prevent the system from detecting dangerous conditions. D: Failure to respond to hazardous CO levels compromises safe HVAC operation.	The dashboard displays a warning when CO exceeds 9.0 ppm. The manual states that hazardous CO conditions trigger an automatic HVAC shutdown.